Chuleta de Python: tipos y métodos

Todo lo básico en una cara · Ingeniería de Datos e IA

🧸 Los 5 tipos básicos

`int`	número entero: `4`, `-5`
`float`	con decimales: `3.14` (¡punto, no coma!)
`str`	texto, entre comillas: `"hola"`, `"4"`
`bool`	verdad o mentira: `True`, `False`
`list`	cesta de cosas: `[4, "a", True]`

⚠️ `4` y `"4"` **no son lo mismo**: uno es un número y el otro es un 4 escrito en un papel.

¿No sabes qué tipo es algo? `type(x)`

🔄 Convertir de un tipo a otro

```
int("-5")      # -5
int(3.9)       # 3 (corta, no redondea)
float("2.5")   # 2.5
str(42)        # "42"
list("hola")   # ['h','o','l','a']
int("hola")    # ❌ error
```

🧮 Operaciones con números

`7 + 2` → 9	`7 - 2` → 5
`7 * 2` → 14	`7 / 2` → 3.5
`7 // 2` → 3	división entera
`7 % 2` → 1	resto (¿es par? `n % 2 == 0`)
`7 ** 2` → 49	potencia

⚖️ Comparar (dan True/False)

`==` igual · `!=` distinto · `<` `>` `<=` `>=`

`and` las dos · `or` alguna · `not` lo contrario

⚠️ `=` **guarda** en una caja; `==` **pregunta** si son iguales.

🛠️ Funciones generales

Herramientas para todo, como unas tijeras ✂️. Se escriben `nombre(cosa)`:

`print(x)`	enseña en pantalla
`input()`	lee una línea (¡siempre da texto!)
`len(x)`	cuánto mide: `len("hola")` → 4
`type(x)`	de qué tipo es
`max` / `min` / `sum`	de una lista de números
`range(a,b)`	de a hasta b-1

🐶 ¿Qué es el punto?

Cada tipo tiene **sus propias habilidades** (métodos). Como un perro sabe ladrar y un gato maullar. El punto dice "usa tu habilidad":

```
cosa.metodo()
texto.upper()     lista.append(4)
```

Un método de texto **no** funciona con una lista, y al revés.

⚠️ ¿Cambia o no cambia?

📄 **Textos: NO cambian.** Te dan una fotocopia. Si no la guardas, se pierde:

```
t.upper()        # ❌ t sigue igual
t = t.upper()    # ✅
```

🛒 **Listas: SÍ cambian** ellas solas. No guardes:

```
l.append(4)      # ✅
l = l.append(4)  # ❌ l pasa a ser None
```

📍 Posiciones (texto y lista)

Se cuenta **desde 0**, como los asientos del cine 🎬

```
t = "hola"
t[0]    # "h"
t[-1]   # "a" (el último)
t[1:3]  # "ol" (de 1 a 2)
```

Igual con listas: `[4,-5,7][1]` → -5

🔤 Métodos de texto (str)

`.upper()`	MAYÚSCULAS
`.lower()`	minúsculas
`.strip()`	quita blancos de los bordes
`.replace(a,b)`	cambia a por b
`.count(x)`	cuántas veces sale x
`.find(x)`	posición de x (-1 si no está)
`.startswith(x)`	¿empieza por x?
`.isspace()`	¿es blanco?
`.isdigit()`	¿son cifras? (⚠️ "-5" da False)
`.isalpha()`	¿son letras?

✂️ split y 🧵 join

split parte un texto y da una **lista**, como una barra de chocolate 🍫:

```
"4  -5 7".split()      # ['4','-5','7']
"a,b,c".split(",")     # ['a','b','c']
```

⚠️ Los trozos siguen siendo **texto**: usa `int()`.

join es lo contrario: pega con un pegamento (primero el pegamento):

```
"-".join(['a','b'])    # "a-b"
```

📋 Métodos de lista (list)

`.append(x)`	añade x al final 🛒
`.insert(i,x)`	mete x en la posición i
`.remove(x)`	quita el primer x (un **valor**)
`.pop()`	quita el último y te lo da
`.pop(i)`	quita la **posición** i
`.sort()`	ordena de menor a mayor
`.reverse()`	le da la vuelta
`.count(x)`	cuántas veces sale x
`.index(x)`	en qué posición está x

`x in lista` → ¿está x? (vale también en textos)

🧩 Todo junto

```
trozos = input().split()
nums = []
i = 0
while i < len(trozos):
    nums.append(int(trozos[i]))
    i = i + 1
print(nums)
# "4 -5 7" → [4, -5, 7]
```

🔍 ¿Dudas? `dir("")`, `dir([])`, `help(str.split)`